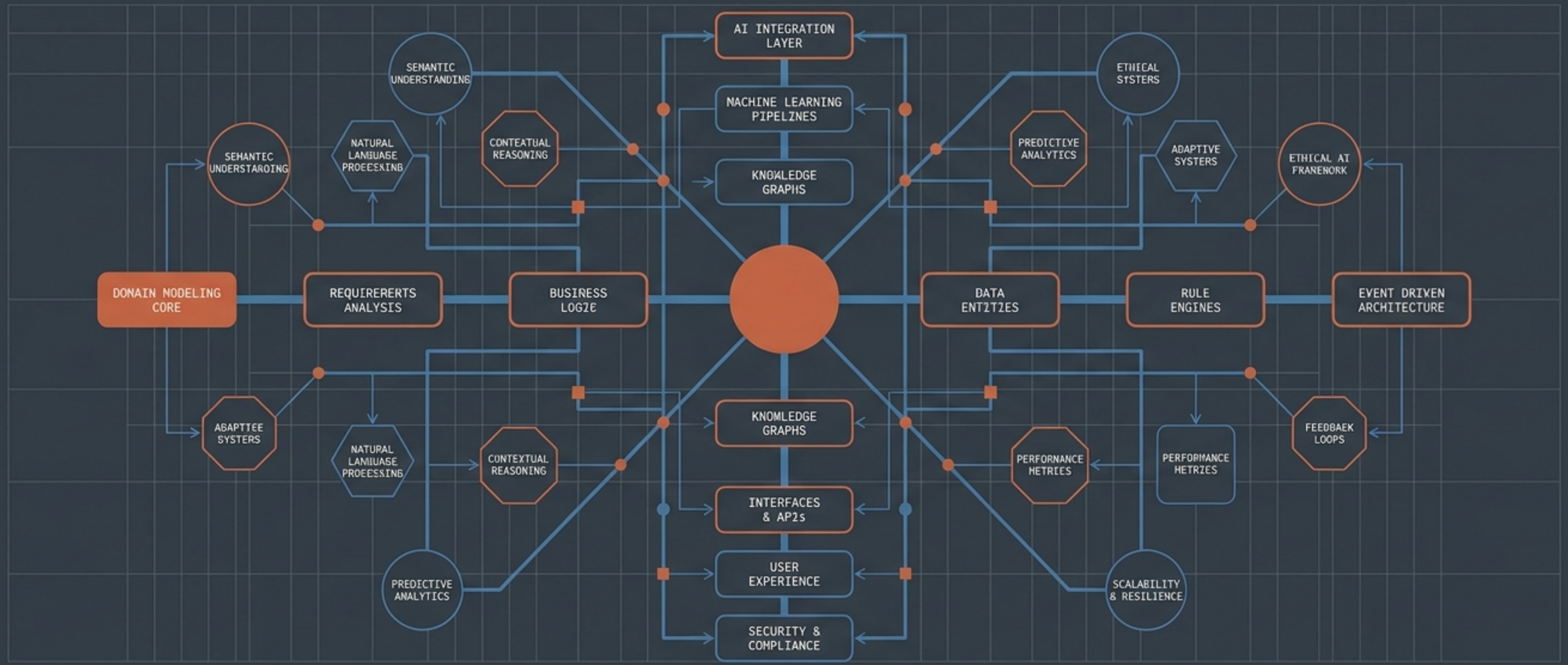
The Architecture of Understanding

Requirements, Domain Modeling, and the AI Era.



Gemini Notebook

The fundamental bottleneck is not documentation.

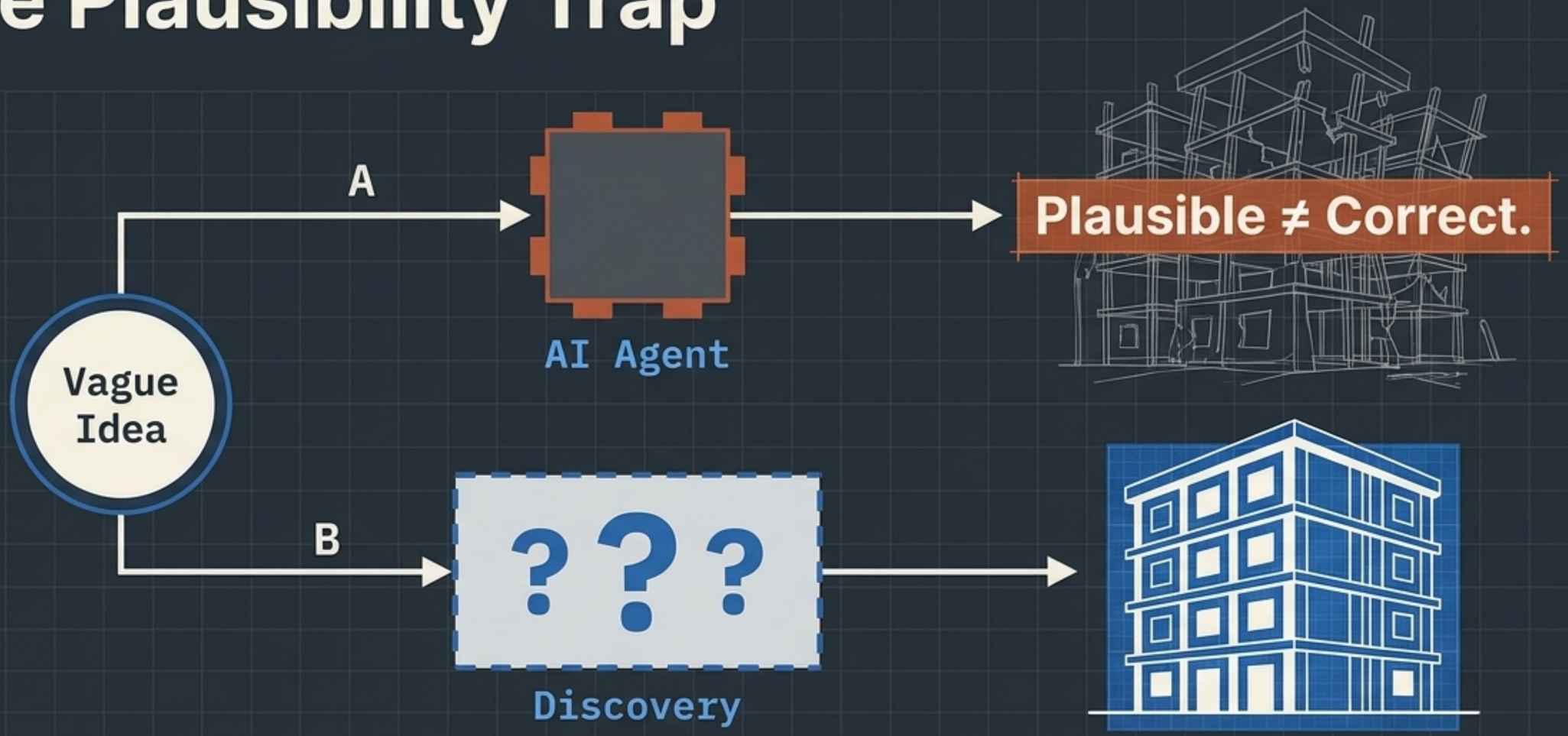
~~CODE~~

UNDERSTANDING

Software doesn’t start with code. No one knows exactly what they want. Before a single line is written, we must answer: What does the system do? For whom? What are the rules? How do we know we are done?

Gemini Notebook

The Plausibility Trap



As code generation becomes virtually free, the cost of generating the wrong code skyrockets. AI knows thousands of systems; it does not know the rules of your specific domain.

Gemini Notebook

Separating the map from the territory

Discovery	Specification
What do we actually need?	What did we agree to build?
■ Conversing with users ■ Exploring examples ■ Discovering rules ■ Finding exceptions ■ EventStorming	■ Domain models ■ User stories ■ Acceptance criteria ■ Engineering tickets

Gemini Notebook

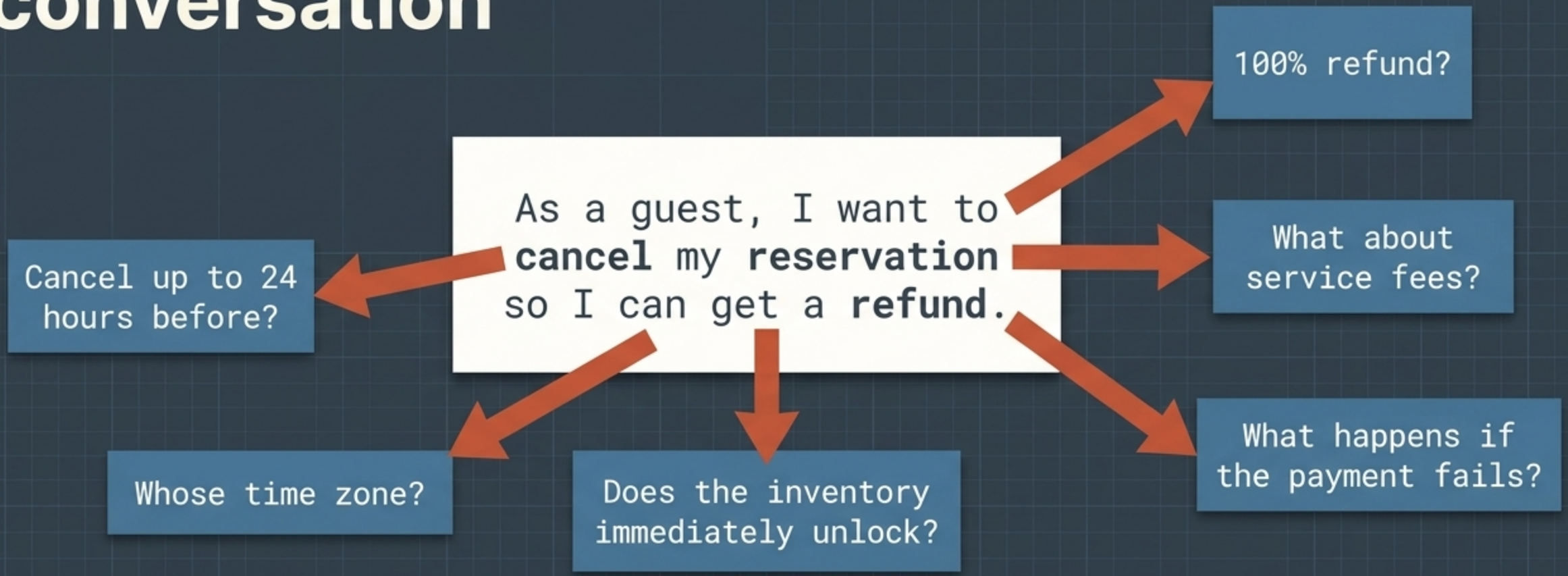
The anatomy of precision

Search page must return results [Functional: Behavior] in < 500ms [Non-Functional: Metric] for 95% of requests [Boundary] at 1000 req/sec [Operating Condition]

ISO standards demand verifiable precision. A system that functions correctly but is unusably slow does not satisfy the business need.

Gemini Notebook

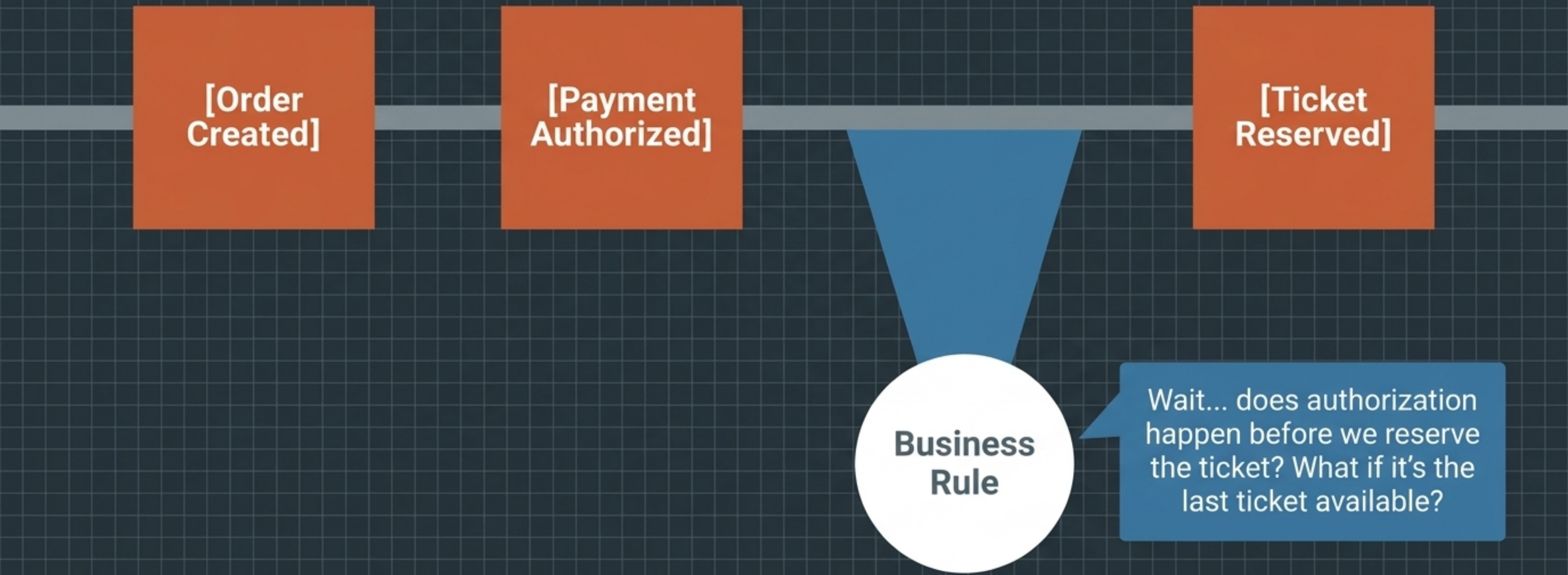
A user story is an invitation to a conversation



If you cannot define the concept, you do not know how to implement it.

Gemini Notebook

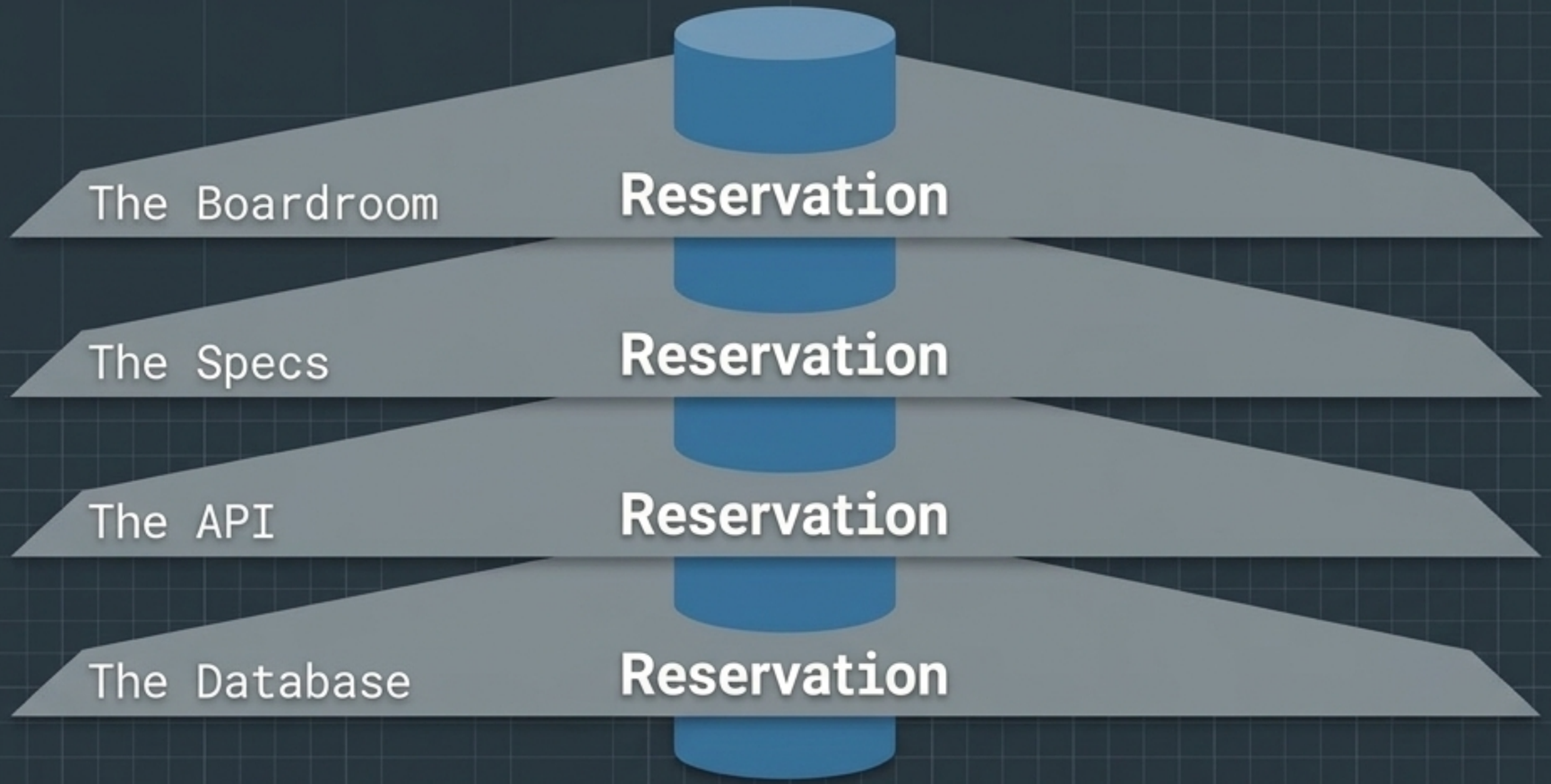
Mapping time to uncover hidden rules



EventStorming is not Event Sourcing. It is an exploration of domain behavior, entirely independent of your database archit

Gemini Notebook

The linguistic bridge to the codebase



Avoid

~~Booking~~

~~Trip~~

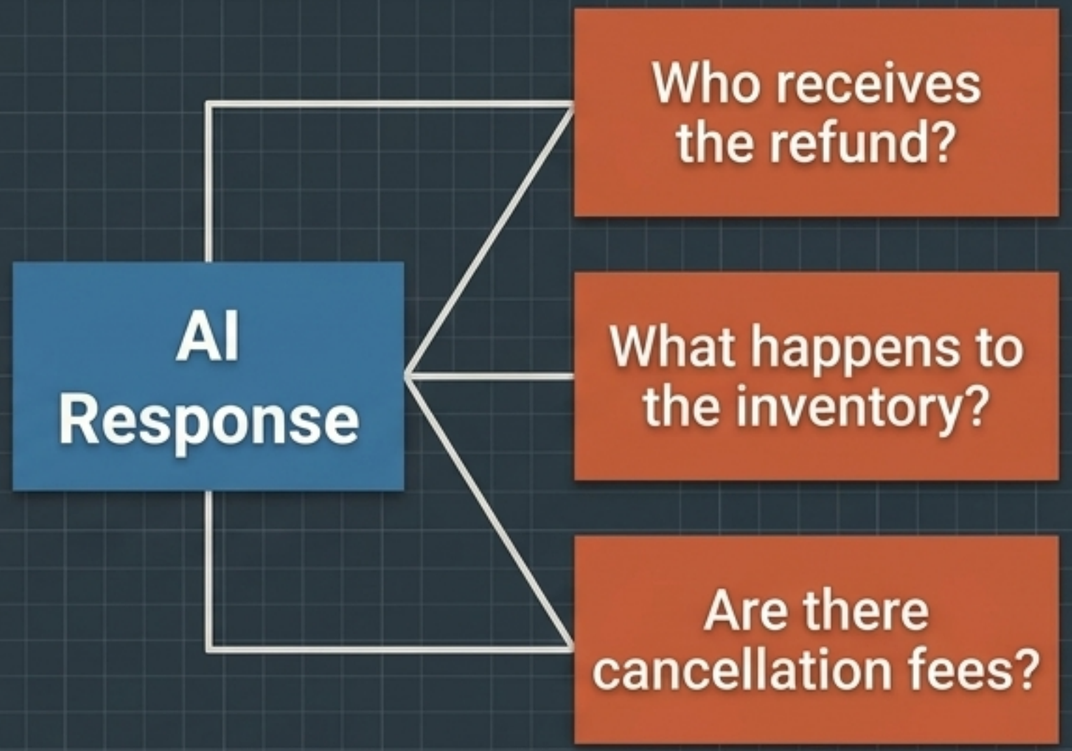
~~Stay~~

Ubiquitous Language is not a decorative glossary. It demands choosing a canonical term and violently rejecting synonyms across the entire lifecycle.

Gemini Notebook

Flipping the AI paradigm: The /grill-me workflow

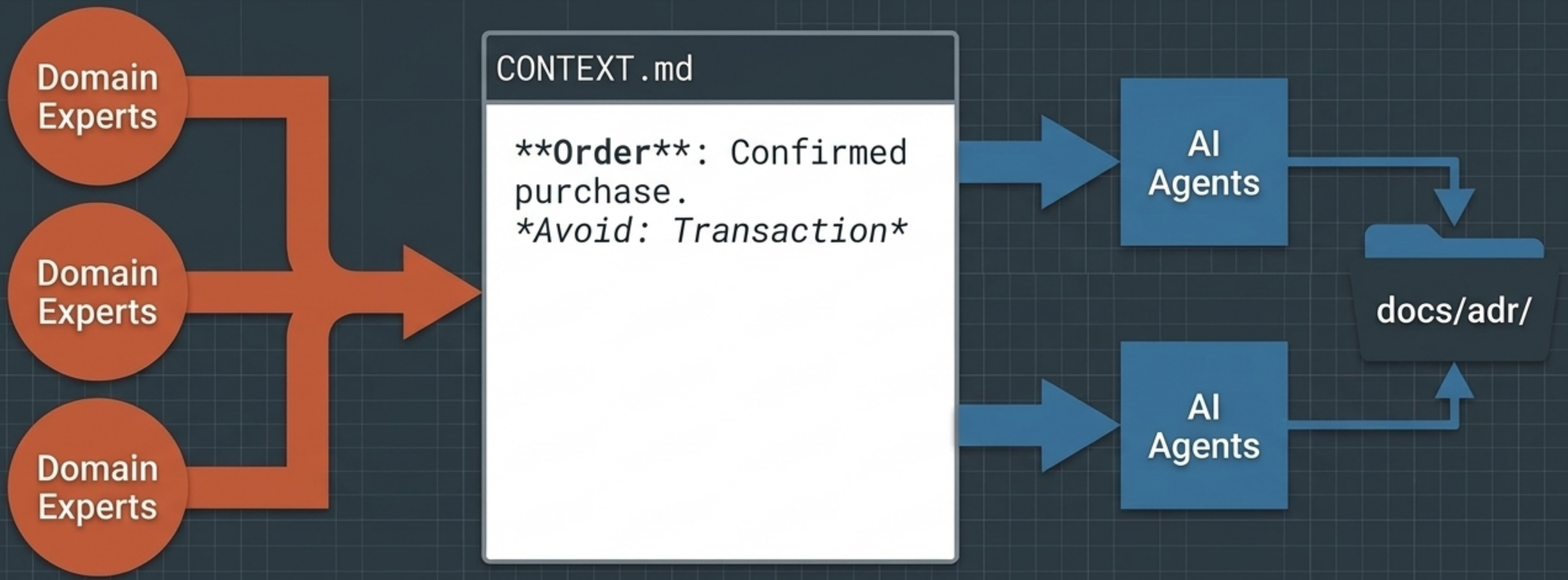
```
Human Prompt:
I want to allow users to
cancel reservations.
```



Transform the AI from a junior coder into a relentless Domain Expert.

Gemini Notebook

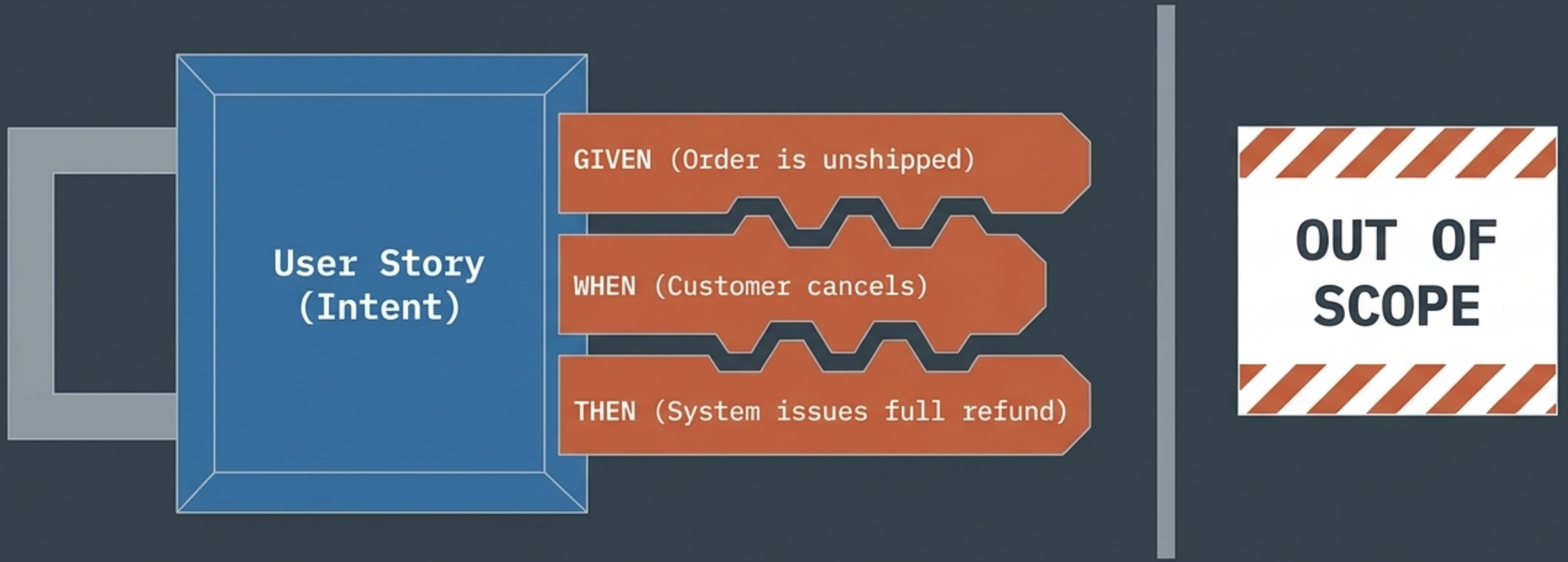
Context as Code



Using /grill-with-docs forces the AI to register the Ubiquitous Language and architectural constraints before a single line of code is written.

Gemini Notebook

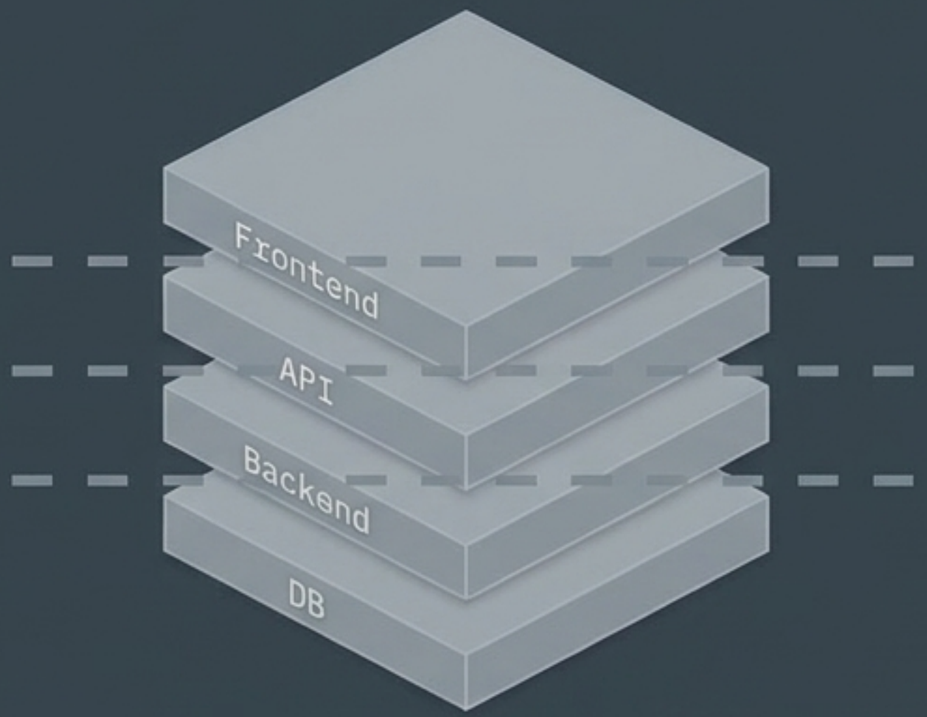
Translating intent into verifiable constraints



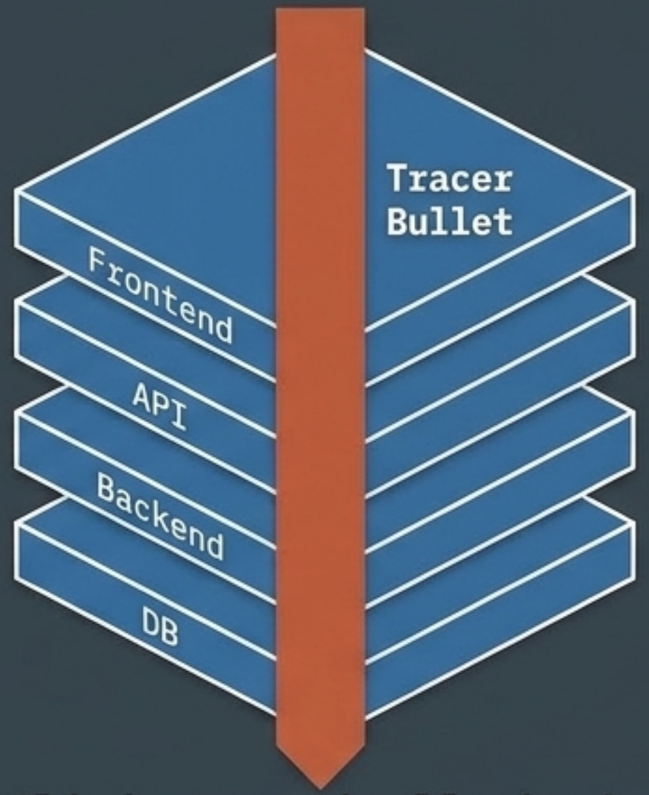
Using /to-spec. Acceptance criteria (Given/When/Then) prove the AI understood the requirement. Explicitly defining what is out of scope prevents AI agents from hallucinating 'helpful' but unwanted features.

Gemini Notebook

Slicing behavior, not technical layers



Wrong: Slicing horizontally by layer.



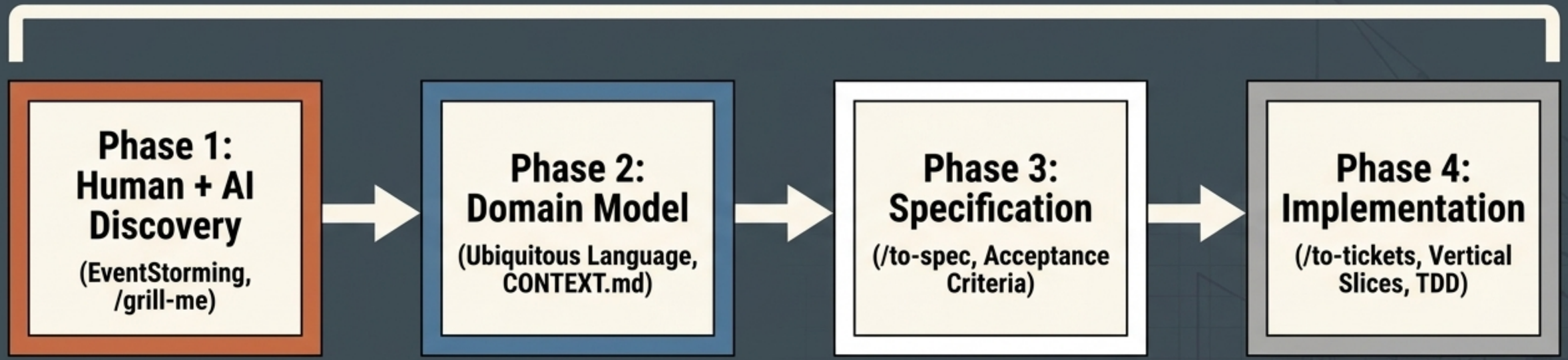
Right: Slicing vertically by behavior.

Using /to-tickets. True AI acceleration requires small, demonstrably complete paths through the entire system. One ticket = one vertical slice of verifiable user behavior

Gemini Notebook

The Modern Engineering Pipeline

AI accelerates every step.



AI can accelerate execution, but it cannot decide what the business actually means.

Gemini Notebook

The Workflow Paradigm Shift

The Past (Manual)	Write massive specs	→ Developers type code	→ Find ambiguity in production.
The Present (The AI Trap)	Vague Idea	⇝ AI instantly generates code	⇝ Structural Chaos.
The Future (Collaborative Modeling)	AI-Assisted Discovery	→ Strict Ubiquitous Language (CONTEXT.md)	→ AI generates targeted vertical slices.

Gemini Notebook

When the language is confused,
stop the code. Model the domain.

Gemini Notebook